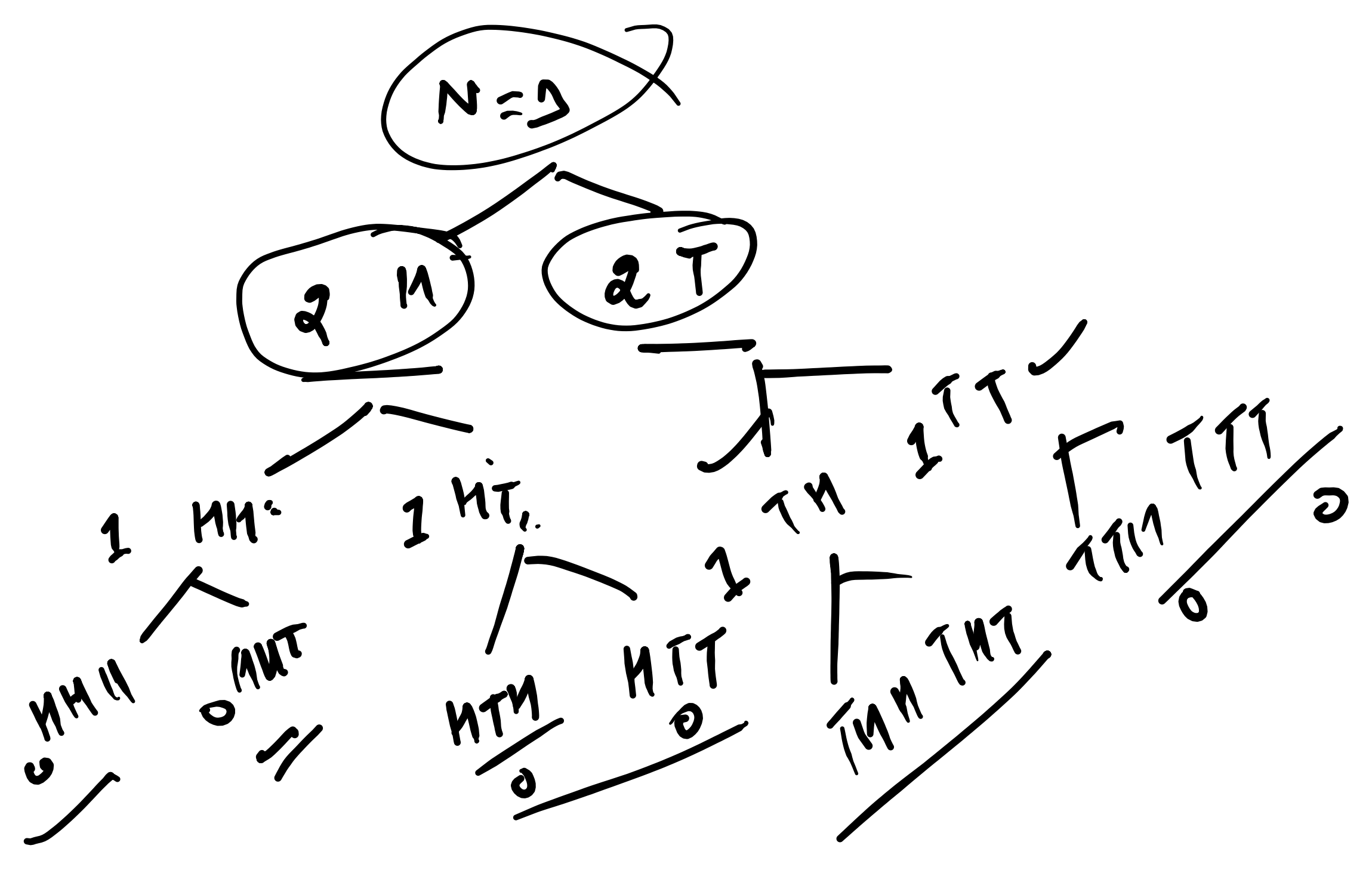
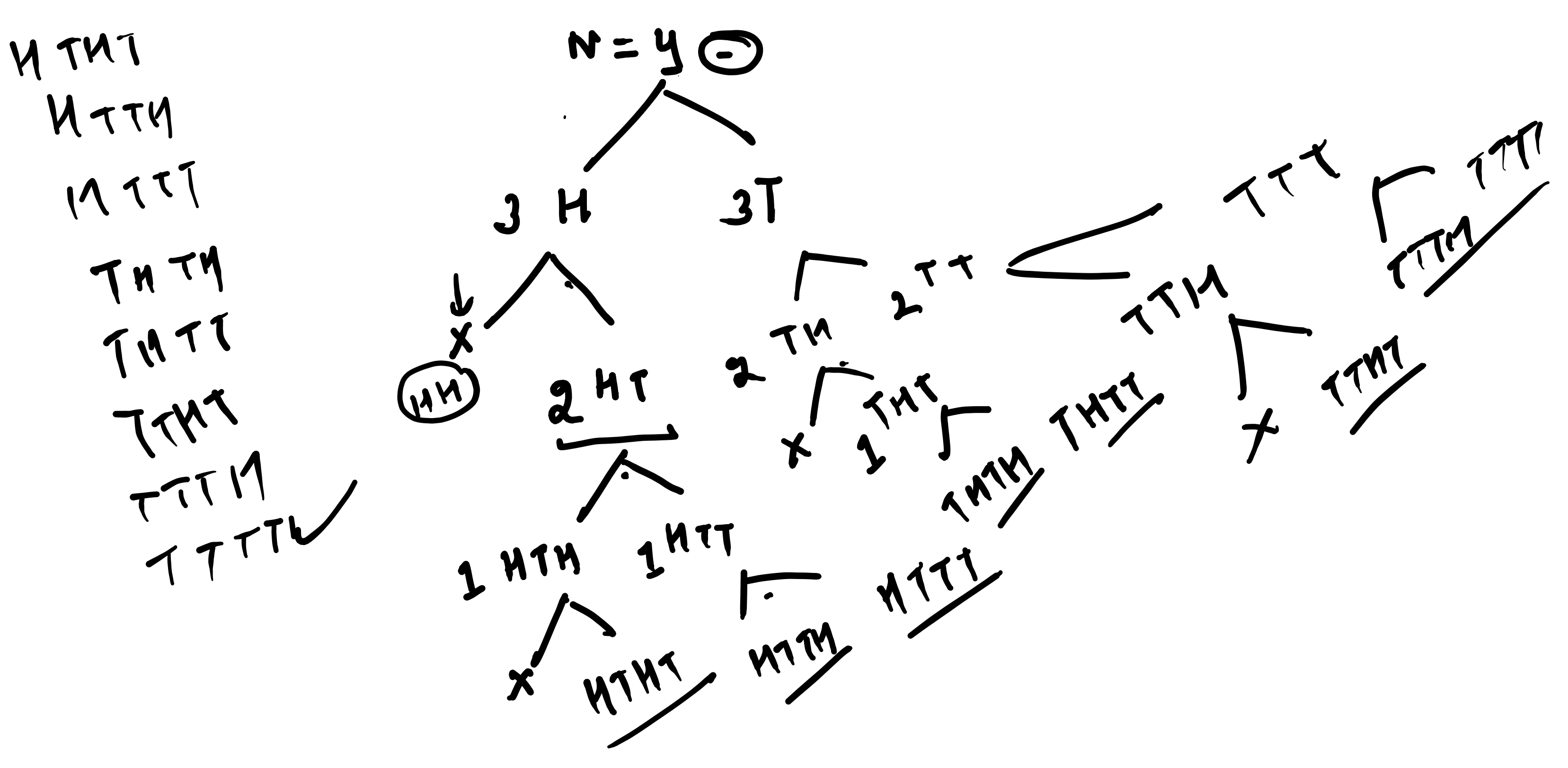
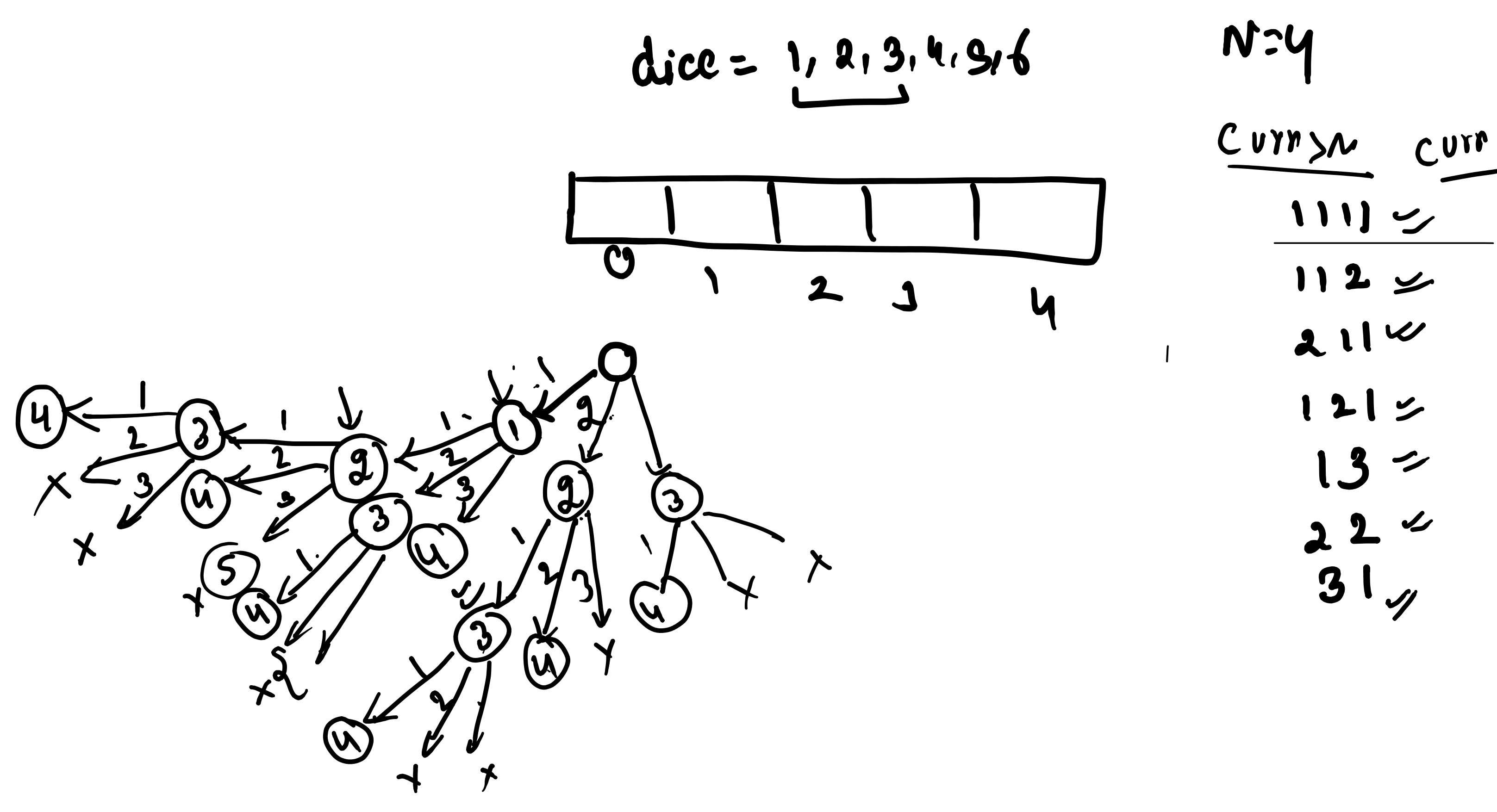
Lec-15

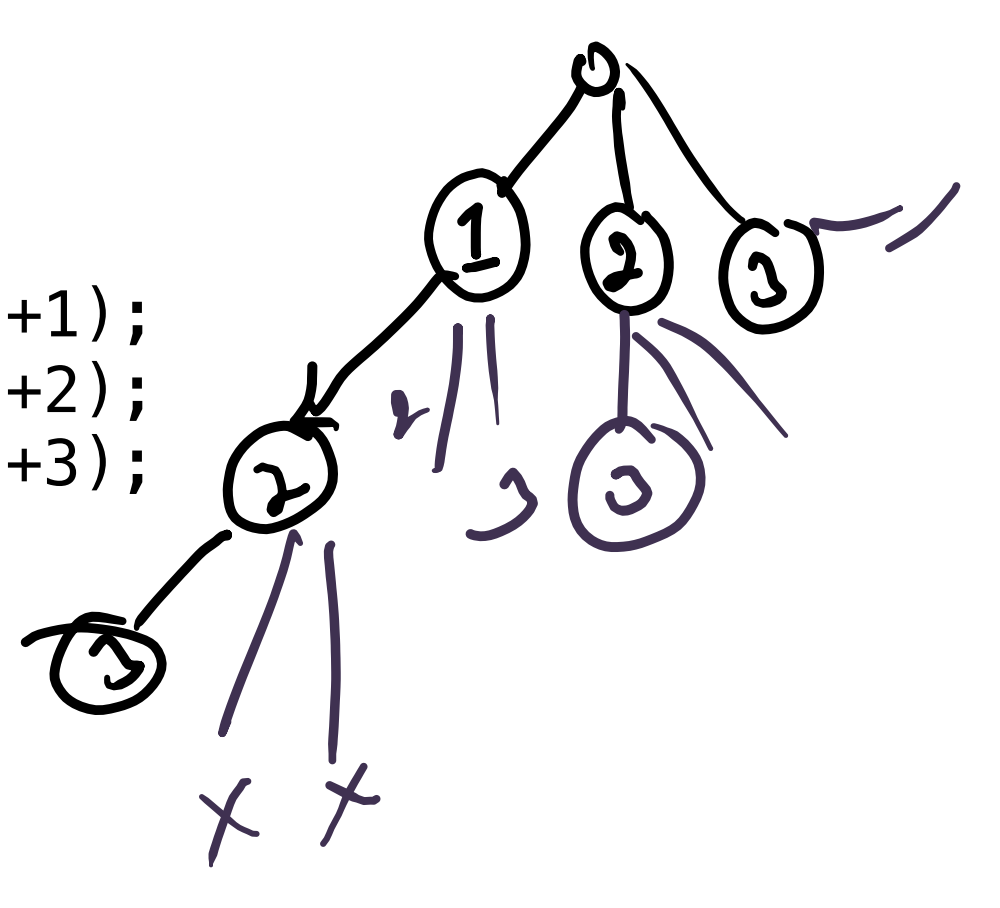
14/10/26 2:09 PM

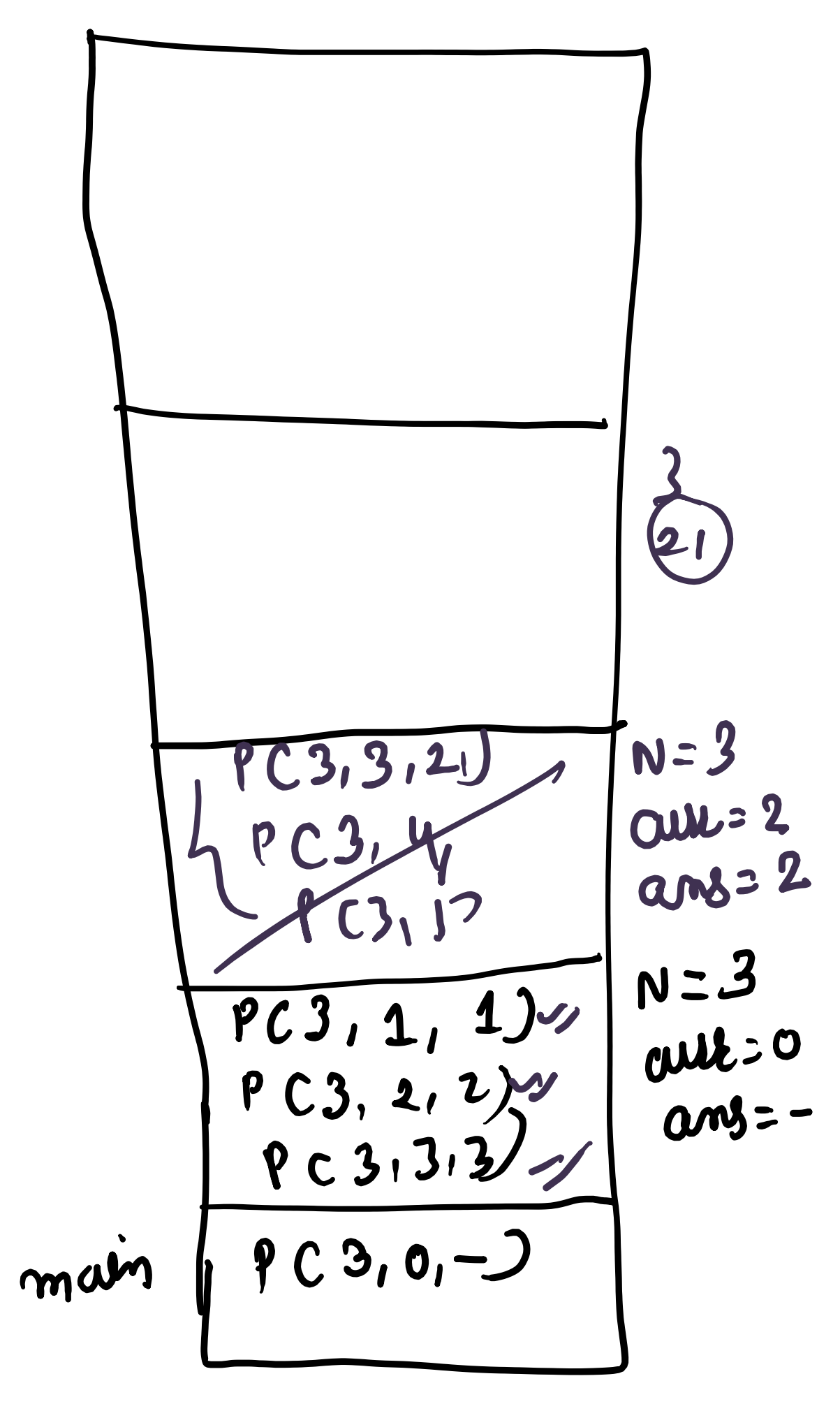




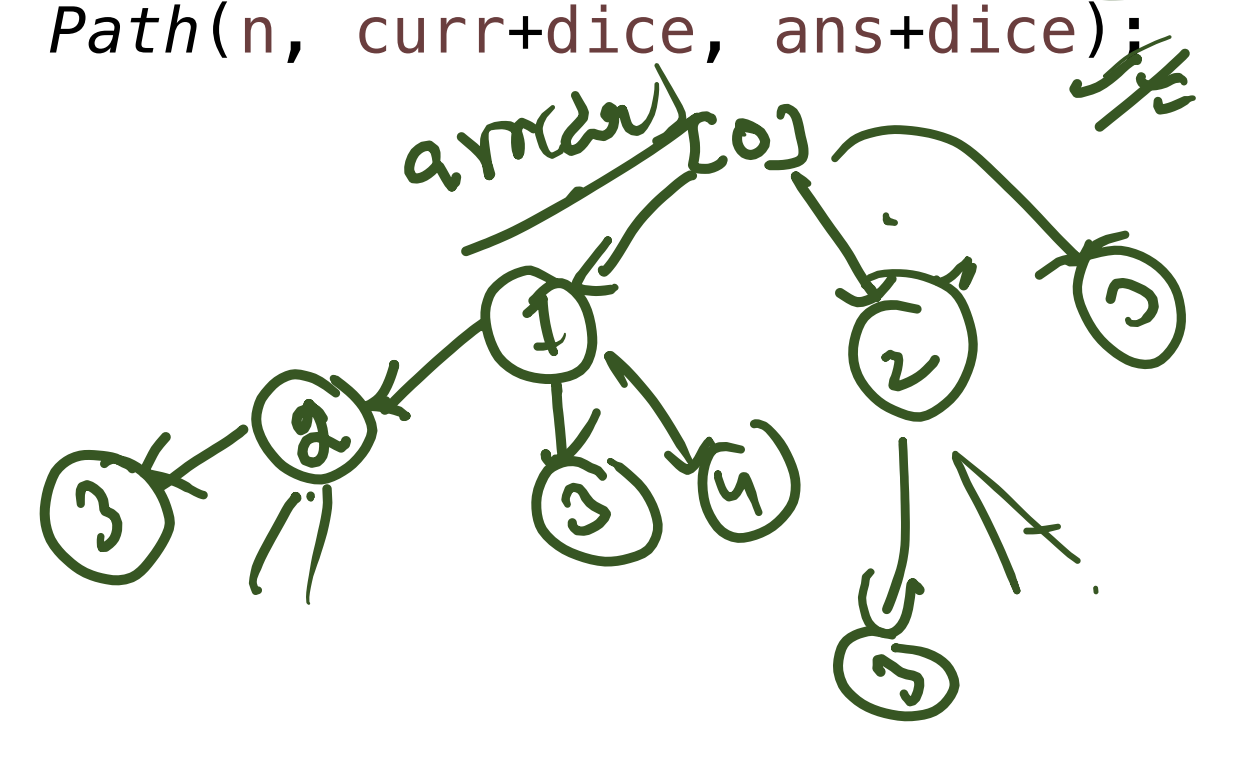


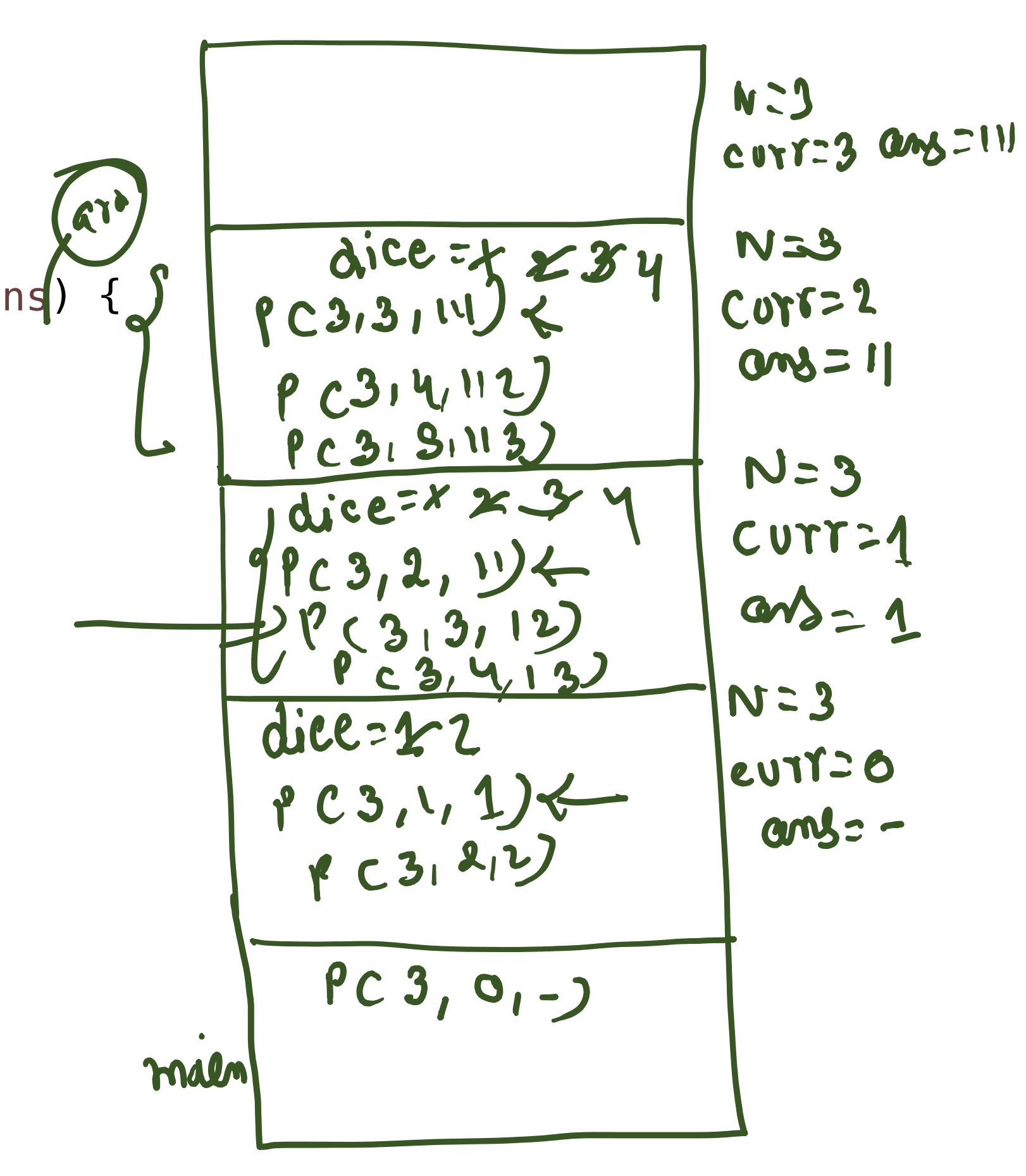
```
public static void main(String[] args) {
        // TODO Auto-generated method stub
        int n = 3;
        Path(n, 0, "");
    }
    public static void Path(int n ,int curr, String ans) {
        if(curr==n) {
            System.out.println(ans);
            return;
        }
        if(curr>n) {
            return;
        }
        Path(n, curr+1, ans+1);
        Path(n, curr+2, ans+2);
        Path(n, curr+3, ans+3);

    }
```



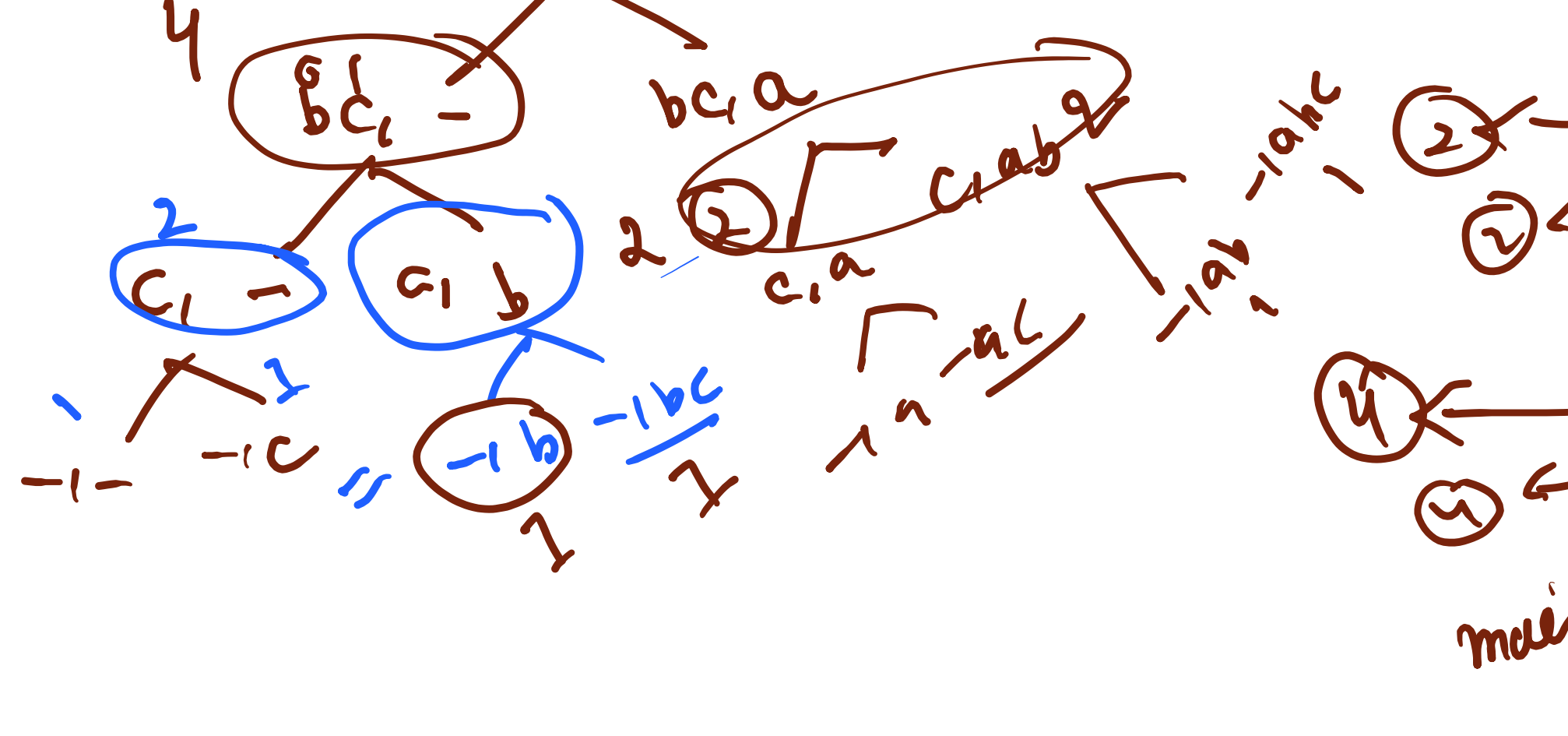


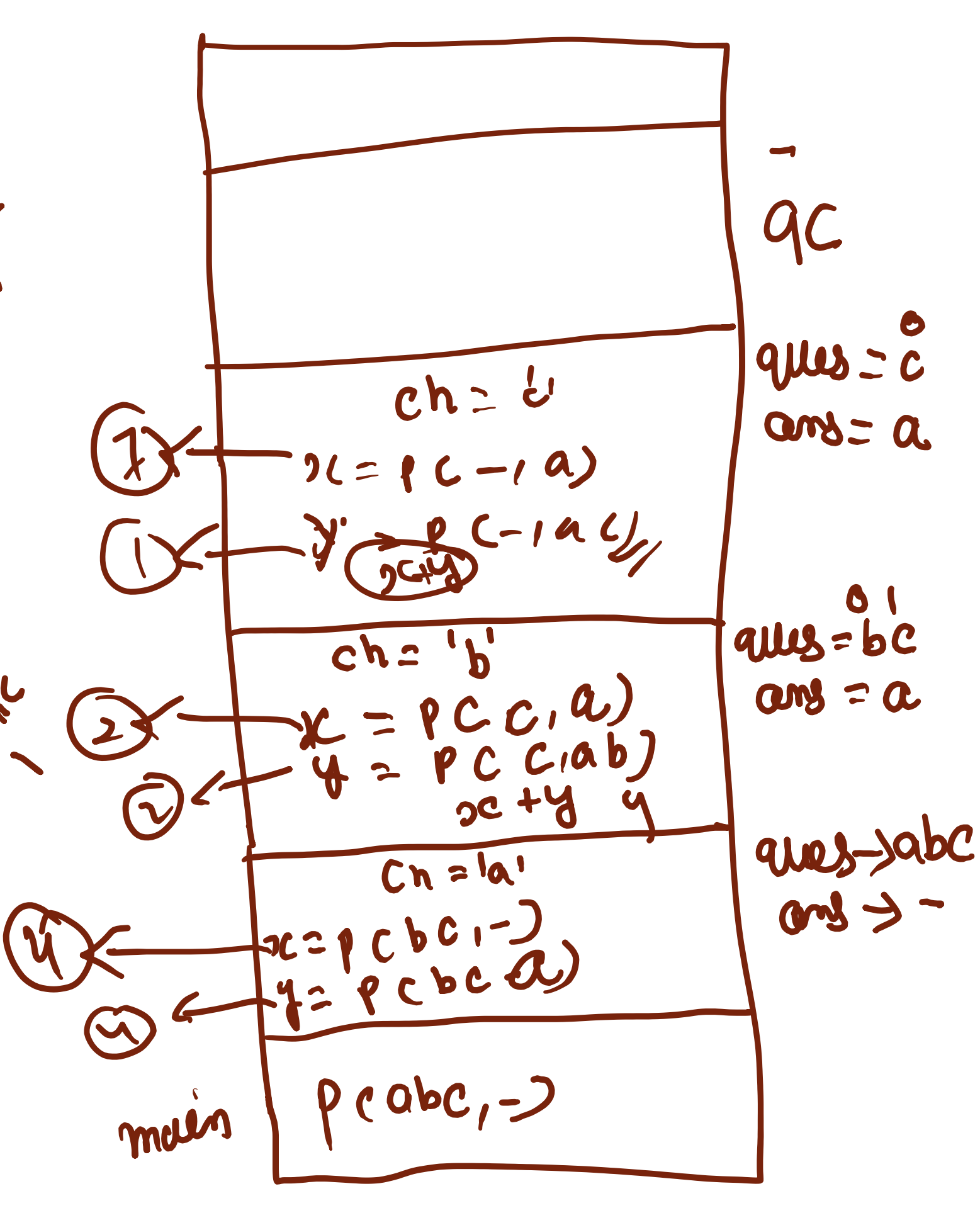
```
public static void main(String[] args) {
        // TODO Auto-generated method stub
        int n = 3;
        Path(n, 0, "");
    }
    public static void Path(int n ,int curr, String ans) {
        if(curr==n) {
            System.out.println(ans);
            return;
        }
        if(curr>n) {
            return;
        }
        for (int dice = 1; dice <=3; dice++) {
            Path(n, curr+dice, ans+dice);
        }

    }
```





```
public static int Count(String ques, String ans) {
        // TODO Auto-generated method stub
        if (ques.length() == 0) {
            System.out.print(ans + " ");
            return 1;
        }
        char ch = ques.charAt(0);
        int x = Count(ques.substring(1), ans);
        int y = Count(ques.substring(1), ans + ch);
        return x + y;

    }
```





Given `n` pairs of parentheses, write a function to *generate all combinations of well-formed parentheses.*

Example 1:

```
Input: n = 3
Output: ["((()))","(()())","(())()","()(())","()()()"]
```